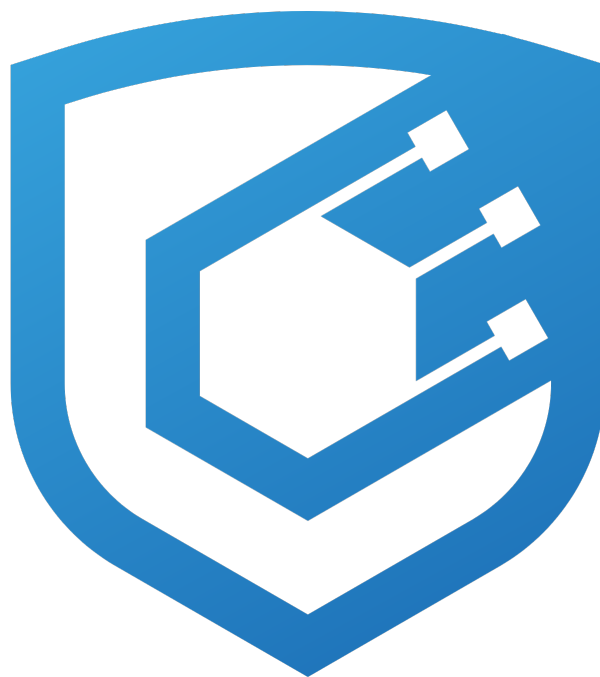




GreeKFi Oracle Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Immeas](#)

[Alix40](#)

September 8, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Centralization Risks	3
6.2	Operational risks	3
7	Findings	5
7.1	Medium Risk	5
7.1.1	Post-deadline Receipt underpricing can cause premature liquidations	5
7.1.2	Receipt collateral has no liquidation exit before <code>exerciseDeadline</code> , letting a matched borrower push bad debt onto Morpho lenders	5
7.2	Low Risk	10
7.2.1	<code>DeployMorphoVault</code> sets the ETH feed max age equal to its heartbeat, so <code>OracleReceipt::price</code> reverts for a short window after most rounds	10
7.3	Informational	11
7.3.1	<code>OracleReceipt::price</code> wraps oversized call moneyness into a negative signed value	11
7.3.2	<code>OracleReceipt</code> retains an unused commented-out <code>SCALE</code> declaration	11
7.3.3	The public Black-Scholes helper does not document its parameter units	11
7.3.4	<code>OracleReceipt::price</code> can publish zero at its moneyness precision boundary	11

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Greek.fi is a fully collateralized, oracle-free options protocol. Each market is an ERC20 pair of an `Option` (the long right to buy or sell collateral at a strike) and a `Receipt` (the short receipt for deposited collateral): users mint both sides 1:1 against collateral, an `Option` holder exercises by paying strike-denominated consideration to receive collateral, and a `Receipt` holder redeems consideration-first then collateral 1:1 on a purely time-gated schedule with no oracle ever consulted.

`OracleReceipt` is an experimental, proof-of-concept lending integration on top of that core: the options protocol itself remains oracle-free, and the oracle is not required for any option to be minted, exercised, or redeemed. It lets Morpho use a `Receipt` as collateral in a market whose loan token is the `Receipt`'s underlying collateral token by supplying the `Receipt` value in Morpho's $1e36$ price scale, so borrowers can draw that loan token against an unexercised short position.

The mark starts from the pair-burn identity, one `Receipt` plus one `Option` always returns one unit of collateral, and therefore values a `Receipt` as $1 - \text{optionValue}$ using normalized Black-Scholes inputs (K/S , time to exerciseDeadline, volatility, and rate). It ignores pool balances and redemption activity so users cannot move the lending price by minting, exercising, burning, or redeeming. `ChainlinkCrossPriceSource` supplies the spot ratio and rejects non-positive, future, zero-timestamp, or stale feed data. Spot-source changes, volatility updates, and rate changes are owner-controlled and take effect on the next price call.

5 Audit Scope

The audit scope was limited to:

```
contracts/OracleReceipt.sol
```

together with the Morpho market deployment path in `script/DeployMorphoVault.s.sol` that configures it.

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [GreekFi Oracle](#) code provided by [GreekFi](#). The findings consist of 2 Medium & 1 Low severity issues with the remainder being informational.

6.1 Centralization Risks

The `OracleReceipt` owner can replace the spot price source, volatility, and rate with no timelock (`OracleReceipt::setParams`), moving the mark used to price a `Receipt` posted as loan collateral in the Morpho integration.

The `OracleReceipt` owner cannot change the core options protocol: it has no authority over the `Factory`, `Option`, or `Receipt` contracts, and the options themselves continue to settle without consulting the oracle. The centralization exposure is confined to the mark used by the lending integration.

6.2 Operational risks

The oracle is an experimental proof of concept, and the Morpho market parameters it is deployed with are part of its risk surface. `DeployMorphoVault` caps expiration at 45 days but does not cap the exercise window. Since oracle pricing and collateral redemption run until `exerciseDeadline`, deployments with longer windows must be reviewed against the full time horizon.

Summary

Project Name	GreekFi Oracle
Repository	contracts
Commit	65f9843ecd9a...
Fix Commit	23bfee78157f...
Audit Timeline	Aug 31st - Sep 2nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	1
Informational	4
Gas Optimizations	0
Total Issues	7

Summary of Findings

[M-1] Post-deadline Receipt underpricing can cause premature liquidations	Resolved
[M-2] Receipt collateral has no liquidation exit before <code>exerciseDeadline</code> , letting a matched borrower push bad debt onto Morpho lenders	Partially Resolved
[L-1] <code>DeployMorphoVault</code> sets the ETH feed max age equal to its heartbeat, so <code>OracleReceipt::price</code> reverts for a short window after most rounds	Resolved
[I-1] <code>OracleReceipt::price</code> wraps oversized call moneyness into a negative signed value	Resolved
[I-2] <code>OracleReceipt</code> retains an unused commented-out <code>SCALE</code> declaration	Resolved
[I-3] The public Black-Scholes helper does not document its parameter units	Resolved
[I-4] <code>OracleReceipt::price</code> can publish zero at its moneyness precision boundary	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Post-deadline Receipt underpricing can cause premature liquidations

Description: `OracleReceipt::price()` always values a Receipt as $1 - \text{optionPrice}$. After `exerciseDeadline`, it sets time to zero but continues valuing the expired exercise right at its current intrinsic value.

That is wrong for an unexercised Receipt. Assignment is closed, and when `consBacked == 0`, each Receipt is a fixed 1:1 collateral claim regardless of later spot movements.

A borrower can create this state by retaining the Options and supplying only the Receipts to Morpho. After the deadline, a spot movement can reduce the reported Receipt price even though the Receipts remain fully collateral-backed.

Morpho can then liquidate an adequately backed position using the incorrect price. An external liquidator can redeem the seized Receipts at par, transferring value from the borrower. Lenders are only exposed if the price gaps past the solvent liquidation range or liquidators fail to act.

Impact: The incorrect price can cause premature liquidation and loss for the borrower. With gradual price movement and active liquidators, the position becomes liquidatable while its collateral can still repay the debt in full. Lender loss requires a sufficiently large price gap or unavailable liquidators.

Recommended Mitigation: Return par after `exerciseDeadline` when `consBacked == 0`, without consulting the live spot price.

GreekFi: Fixed in [PR38](#).

Cyfrin: Verified. The oracle now returns par after the exercise deadline when `consBacked == 0`, so fully unexercised Receipts no longer follow the live spot price. Partially assigned series retain the documented conservative settlement mark as residual liquidation risk.

7.1.2 Receipt collateral has no liquidation exit before `exerciseDeadline`, letting a matched borrower push bad debt onto Morpho lenders

Description: Morpho lenders are protected only by liquidation. When a position falls below the threshold, a keeper repays the debt, seizes the collateral, and profits from the liquidation incentive ($\text{LIF} = \min(1.15, 1 / (1 - 0.3 * (1 - \text{LLTV})))$, 1.0438 at the deployment's 86% LLTV). The loan token is sUSDe and the seized collateral is Receipt tokens, so a keeper only liquidates if it can turn those Receipts into sUSDe in the same flow. There are only three ways a Receipt becomes an underlying token, and there is no DEX for them:

- `Option.burn` (pair-burn) pays sUSDe 1:1 and is open at any time, but it needs the matching `Option` tokens.
- `Receipt.redeem` before `exerciseDeadline` pays WETH, not sUSDe, and only up to `consBacked`, a single global first-come pool. The keeper then still has to sell that WETH for sUSDe, right after the move that triggered the liquidation.
- The collateral leg of `redeem` pays sUSDe 1:1, but only once `block.timestamp > exerciseDeadline`.

`Receipt::_redeem` shows the last two. The consideration leg is capped at `consBacked` (zero until someone exercises) and the collateral leg is gated on the deadline, so while a series is unassigned every `redeem` reverts `ExerciseWindowOpen`:

```
function _redeem(address account, uint256 amount_) internal nonZero(amount_) {
    if (isEuro() && block.timestamp < expirationDate()) revert BeforeExerciseWindow();
    uint256 amount = amount_ > consBacked ? consBacked : amount_;
    uint256 remaining = block.timestamp > exerciseDeadline() ? amount_ - amount : 0;
    uint256 consAmount = toConsideration(amount, false);
    if (amount == 0 && remaining == 0) revert ExerciseWindowOpen();
    consBacked -= amount;

    if (amount > 0 && consideration().balanceOf(address(this)) < consAmount) revert InsufficientPool();
    if (remaining > 0 && collateral().balanceOf(address(this)) < remaining) revert InsufficientPool();
}
```

```

uint256 rtBurned = amount + remaining;
_burn(account, rtBurned);

uint256 considerationPaid = _payout(consideration(), account, consAmount);
uint256 collateralPaid = _payout(collateral(), account, remaining);
emit Redeemed(msg.sender, account, rtBurned, collateralPaid, considerationPaid);
}

```

So calling `liquidate` is not enough. A keeper needs path 1 or 3, or a raced slice of 2. A stranger who seizes the Receipts has none of these before the deadline. It holds no `Option`, so path 1 is closed. Path 3 is gated until the deadline. Path 2 pays the wrong asset, is capped at `consBacked`, and reverts while `consBacked` is zero. Off-chain there is no dependable venue either. Receipts trade only through discretionary request-for-quote market making, and a maker who buys a seized Receipt inherits the same lockup, so it will not quote near the mark for a bespoke series or during the move that triggered the liquidation. (It is also important to note, liquidation bot usually require onchain solutions to execute liquidations and not off-chain)

The minter who kept the `Option` tokens is the exception. `Option.burn` pair-burns a Receipt with its `Option` for sUSDe 1:1 at any time:

```

// Option
function burn(address account, uint256 amount) public nonReentrant nonZero(amount) {
    if (notAuthorized(account, msg.sender, Perm.BURN)) revert Unauthorized();
    _burn(account, amount);
    receipt.burn(account, amount);
    emit PairBurned(msg.sender, account, amount);
}

// Receipt
function burn(address account, uint256 amount) public onlyOption nonReentrant {
    _burn(account, amount);
    collateral().safeTransfer(account, amount);
}

```

So the minter self-liquidates. It supplies the Receipts to Morpho, keeps the Options, borrows sUSDe, and waits. Once a normal price move drops the mark below $LLTV * LIF$ (0.898 of the mark at borrow time), it calls `liquidate` on its own position, receives the Receipts in the callback, pair-burns them with the retained Options for par sUSDe, repays only $collateral * mark / LIF$, and Morpho writes off the rest. An isolated mint of any size is the whole proof, and a roughly 20% adverse move by mid-life crosses the threshold.

The bug does not depend on the mark being wrong. Whether or not the mark is exactly right, no stranger can convert a live, unassigned Receipt into the sUSDe borrow token, while the matched minter can. The loss comes from the collateral being unrealisable by anyone else, not from a pricing error.

This survives the objection that an ETH dump makes the puts exercise, which raises `consBacked` and opens path 2 for keepers. That does not save lenders when the borrower is also the dominant writer. Suppose one wallet holds most of the Receipts, supplied to Morpho, and the matching Options, and does not sell the longs. Anyone can call `liquidate` once the mark is below the threshold, but nobody can realise that wallet's Receipts as sUSDe. Outsiders hold only the minority of Options, so they can push `consBacked` to at most their own share and cannot assign the wallet's position. A stranger who seizes the wallet's Receipts can redeem at most that minority as WETH, and only if those outsiders actually exercise and the free Receipt holders do not drain the pool first. If the free holders take the WETH pool, `consBacked` returns to zero and the seized Receipts are fully stuck. The wallet is the only party that can burn them for sUSDe, so it self-liquidates and the write-off is lender loss.

The loss scales with the market's supply, not the 1,000 sUSDe cap, which only limits the MetaMorpho vault. Direct Morpho suppliers are uncapped, and the borrower can loop mint, supply, and borrow for leverage of about 4.6x at the ATM 30-day mark used above, up to roughly 7x when the mark is near par. Among MetaMorpho v1.1 vault depositors the loss is not even shared, because the vault does not mark shares down on a market loss, so shares redeem at par until the vault runs dry and the last depositor to redeem absorbs the shortfall. With two 500-sUSDe depositors and the 18.72 sUSDe write-off from the 20% path above, the first to redeem gets out whole and the last one absorbs the 18.72. Direct Morpho Blue suppliers, by contrast, socialise the loss pro-rata.

Impact: Morpho lenders lose loan principal as bad debt, because no third party can reliably realise the seized Receipt collateral while the option is live, whereas the matched borrower can always recover par.

Proof of Concept: Add the following file to test/ and run `forge test --match-contract PreDeadlineNoExit -vv`. The first test shows a liquidator stand-in cannot exit while the matched holder pair-burns for par. The other two print the mark path and assert the bad-debt threshold is crossed.

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.30;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { OracleReceiptTest } from "../OracleReceipt.t.sol";
import { Option } from "../contracts/Option.sol";
import { Receipt as Rct } from "../contracts/Receipt.sol";
import { IReceipt } from "../contracts/interfaces/IReceipt.sol";

contract PreDeadlineNoExit is OracleReceiptTest {
    uint256 constant LLTV_WAD = 0.86e18;

    function _lif() internal pure returns (uint256) {
        // Morpho Blue: min(1.15, 1 / (1 - 0.3 * (1 - LLTV)))
        uint256 denom = 1e18 - (0.3e18 * (1e18 - LLTV_WAD)) / 1e18;
        uint256 lif = 1e36 / denom;
        return lif > 1.15e18 ? 1.15e18 : lif;
    }

    function _threshold(uint256 mark0) internal pure returns (uint256) {
        return mark0 * LLTV_WAD / 1e18 * _lif() / 1e18;
    }

    function test_PreDeadline_ThirdPartyCannotRealise_MatchedHolderCan() public {
        uint256 amount = 1_000e6;
        IReceipt receipt = putOracle.r();
        Option putOption = Option(address(receipt.option()));
        Rct rct = Rct(address(receipt));

        // attacker = this: mints the matched pair, keeps the Options, hands the receipts to a
        // ↪ liquidator stand-in
        usdc.mint(address(this), amount);
        usdc.approve(address(factory), amount);
        putOption.mint(amount);

        address liquidator = makeAddr("liquidator");
        IERC20(address(rct)).transfer(liquidator, amount);

        // 15 days in, still pre-expiry (expiry = +30d): the liquidator holding receipts cannot exit
        vm.warp(block.timestamp + 15 days);
        assertEq(receipt.consBacked(), 0);
        vm.prank(liquidator);
        vm.expectRevert(IReceipt.ExerciseWindowOpen.selector);
        rct.redeem();
        vm.prank(liquidator);
        vm.expectRevert(IReceipt.ExerciseWindowOpen.selector);
        rct.redeem(1);

        // matched holder: receipts back with the attacker, pair-burn for par at any time
        vm.prank(liquidator);
        IERC20(address(rct)).transfer(address(this), amount);
        uint256 before = usdc.balanceOf(address(this));
        putOption.burn(amount);
        assertEq(usdc.balanceOf(address(this)) - before, amount, "pair-burn returns par pre-deadline");
    }
}
```

```

/// Entry A: borrow at the ATM mark 30 days out, how far must spot fall by mid-life
function test_PreDeadline_EntryAtm30d() public {
    uint256 mark0 = putOracle.price();
    uint256 thr = _threshold(mark0);
    emit log_named_decimal_uint("A mark0 (ATM, 30d)           ", mark0, 36);
    emit log_named_decimal_uint("A bad-debt threshold         ", thr, 36);
    vm.warp(block.timestamp + 15 days);
    uint256[3] memory spots = [uint256(1700e18), 1600e18, 1500e18];
    for (uint256 i; i < 3; ++i) {
        source.setSpot(spots[i]);
        uint256 m = putOracle.price();
        emit log_named_decimal_uint(i == 0 ? "A mark, spot -15%, 15d left " : i == 1 ? "A mark,
            ↪ spot -20%, 15d left " : "A mark, spot -25%, 15d left ", m, 36);
    }
    source.setSpot(1600e18);
    assertLt(putOracle.price(), thr, "20% drop mid-life creates bad debt on a max-LTV position
    ↪ opened ATM");
}

/// Entry B: borrow 2 days before expiry with the put 5% OTM (mark near par), spot then slides into
↪ the deadline
function test_PreDeadline_EntryOtmNearExpiry() public {
    vm.warp(expiry - 2 days);
    source.setSpot(2100e18);
    uint256 mark0 = putOracle.price();
    uint256 thr = _threshold(mark0);
    emit log_named_decimal_uint("B mark0 (5% OTM, 2d)           ", mark0, 36);
    emit log_named_decimal_uint("B bad-debt threshold         ", thr, 36);
    vm.warp(deadline - 1);
    uint256[3] memory spots = [uint256(1900e18), 1800e18, 1700e18];
    for (uint256 i; i < 3; ++i) {
        source.setSpot(spots[i]);
        uint256 m = putOracle.price();
        emit log_named_decimal_uint(i == 0 ? "B mark, spot 0.95K, deadline " : i == 1 ? "B mark,
            ↪ spot 0.90K, deadline " : "B mark, spot 0.85K, deadline ", m, 36);
    }
    source.setSpot(1750e18);
    assertLt(putOracle.price(), thr, "a ~17% slide from 5% OTM to 12.5% ITM into the deadline
    ↪ creates bad debt");
}
}

```

The full end-to-end attack was reproduced against live Morpho on a mainnet fork with the option still live. Borrow 780 sUSDe against a 1,000-collateral position at the ATM 30-day mark (0.9105), warp 15 days so roughly 15 days remain before the deadline, move spot 20% adverse so the mark falls to 0.7957 below the 0.8173 threshold, then self-liquidate. The borrower seizes all 1,000 collateral, pair-burns it with the retained Options for full par in `onMorphoLiquidate`, repays only 762.29, and Morpho writes off 18.72 sUSDe of bad debt against the suppliers, all before `exerciseDeadline`. A third party holding the seized Receipt at that instant reverts `ExerciseWindowOpen` on `redeem`.

Recommended Mitigation: There is no reliable on-chain way for a third party to realise a seized Receipt in the loan token before `exerciseDeadline`, so the practical mitigation is operational. None of the following requires a contract change.

- Keep the markets experimental and curated. Do not open them to public or permissionless lenders.
- Create each lending series with a short maturity. A keeper that warehouses waits from the liquidation to `exerciseDeadline`, so a shorter maturity means less time exposed with no third-party exit. Keep `windowSeconds` small as well, so the deadline stays close to expiry and a series cannot be created with one far past expiry that stretches the wait and inflates the oracle's time-to-expiry. The deployed American puts already use a short window.

- Create the Morpho market with a conservative LLTV, sized to the adverse move you are willing to absorb over the series, since a lower LLTV pushes the bad-debt line $\text{mark0} * \text{LLTV} * \text{LIF}$ further down. Morpho allows only enabled LLTV values and the market is immutable, so this is a creation-time choice. On the PoC's 20% path the enabled 62.5% avoids bad debt where 86% does not, and a deeper crash still breaches it.
- Keep the redemption fee at or below the documented 1% and never raise it on a live market. A fee above the near-par break-even, about 420 bps at 86% LLTV ($1 - 1 / \text{LIF}$), makes a third-party redeem-at-the-mark unprofitable. Pair-burn pays no fee, so a high fee never slows the self-liquidator.
- Rely on a warehousing keeper. At a low LLTV this is profitable enough that a specialist willing to lock capital to the deadline will do it, though ordinary Morpho flash liquidators will not, since they need sUSDe in the same transaction. It liquidates at the first crossing, holds the seized `Receipt` to `exerciseDeadline`, and redeems. The liquidation bonus, about 12.7% at 62.5% LLTV against 4.4% at 86%, is the return if the keeper realises at the mark, and it covers the roughly 1.2% cost of capital over a 45-day wait with wide margin. In the common unassigned case the keeper does better: it pays about mark / LIF (0.81 at mark 0.91 and 62.5%) and redeems the collateral leg at par minus the fee (0.99 at a 1% fee), about a 22% total on the hold. A first-crossing liquidation repays lenders in full, and losing that race to the borrower still leaves lenders whole, since the borrower only takes the bonus. So run or fund a specialist keeper for these markets.
- Monitor position health against the mark, feed freshness, and `consBacked`, so the keeper can act at the first crossing.

Residuals to document for anyone supplying to these markets. The exit still depends on a keeper being present and winning the race to the first crossing, which is not guaranteed for a thin market. A gap through both the liquidation and bad-debt lines in one oracle update, or a stale feed that reverts `price` and freezes liquidation, can still leave bad debt. Keepers need standing capital that a large simultaneous move can exhaust. The fee is an owner setting, so keeping it low relies on operator discipline. Direct Morpho supply is not capped by the vault.

Before any public or permissionless use these markets need a reliable third-party exit, which operations cannot supply and which is a larger design change outside this guidance. Do not rely on a discretionary off-chain options market, and do not use an escrowed-`Option` collateral tier.

GreekFi: Risk is documented and warned against

Cyfrin: Acknowledged. The README now marks `OracleReceipt` and `DeployMorphoVault` as experimental and warns against public lender liquidity and the missing pre-deadline exit, but no on-chain third-party exit was added, so the bad-debt path itself remains and the finding is partially resolved.

7.2 Low Risk

7.2.1 DeployMorphoVault sets the ETH feed max age equal to its heartbeat, so OracleReceipt::price reverts for a short window after most rounds

Description: DeployMorphoVault wires ChainlinkCrossPriceSource with UNDERLYING_MAX_AGE = 3600 for the ETH/USD feed, and the comment describes the value as "heartbeat + grace":

```
address constant ETH_USD_FEED = 0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419;
address constant SUSDE_USD_FEED = 0xFF3BC18cCBd5999CE63E788A1c250a88626aD099;
uint256 constant UNDERLYING_MAX_AGE = 3600; // heartbeat + grace
uint256 constant CASH_MAX_AGE = 90000; // 25h
```

```
ChainlinkCrossPriceSource source = new ChainlinkCrossPriceSource(
    AggregatorV3Interface(ETH_USD_FEED), AggregatorV3Interface(SUSDE_USD_FEED), UNDERLYING_MAX_AGE,
    ↪ CASH_MAX_AGE
);
```

3600 s is exactly the ETH/USD heartbeat, with no grace. Chainlink's heartbeat round lands a few seconds late in calm markets, and ChainlinkCrossPriceSource::_read rejects any answer older than maxAge with a strict comparison:

```
function _read(AggregatorV3Interface feed, uint256 maxAge) internal view returns (uint256) {
    (, int256 answer, , uint256 updatedAt,) = feed.latestRoundData();
    if (answer <= 0) revert InvalidPrice(address(feed));
    if (updatedAt == 0 || updatedAt > block.timestamp || block.timestamp - updatedAt > maxAge) {
        revert StalePrice(address(feed));
    }
    return uint256(answer);
}
```

Sampled on mainnet on 2026-09-01 from the live aggregator behind the ETH/USD proxy (rounds 33173 to 33193): 11 of the last 20 inter-round gaps were 3612 to 3636 s. During the 12 to 36 s after each such round `block.timestamp - updatedAt > 3600` holds and every `price()` call reverts `StalePrice`. The sUSDe/USD leg is configured correctly (24 h heartbeat, 25 h max age).

Impact: Morpho borrow, liquidate, and debt-side withdrawCollateral revert for a 12-36 second window after most ETH/USD heartbeat rounds.

Recommended Mitigation: Set the underlying max age to the heartbeat plus a real grace period and make the comment true, for example:

```
uint256 constant UNDERLYING_MAX_AGE = 3600 + 600; // heartbeat + grace
```

Chainlink's guidance is to allow a margin above the heartbeat. A value of 3900 to 4200 s keeps the staleness guard meaningful without tripping on ordinary round latency.

GreekFi: Fixed in [PR40](#)

Cyfrin: Verified. The deployment script now allows a ten-minute grace period above the ETH/USD feed heartbeat.

7.3 Informational

7.3.1 `OracleReceipt::price` wraps oversized call moneyness into a negative signed value

Description: `Factory::createOption2` permits an arbitrarily large nonzero strike when the consideration token has no more decimals than the collateral token. If an integration attaches `OracleReceipt` to such a call market, `OracleReceipt::price` can compute moneyness above `type(int256).max`. The pure pricing boundary then converts that unsigned value to `int256`, wrapping it to a negative number before the logarithm and intrinsic-value calculations.

Impact: Pricing a deliberately extreme call market can revert while the market is live and can return zero or revert after the exercise deadline, depending on the wrapped value. This fails closed rather than overvaluing collateral, and the documented Morpho deployment accepts only puts, so the supported production profile is not exposed. Generic integrations that use `OracleReceipt` for call markets can nevertheless receive a broken mark.

Recommended Mitigation: Reject moneyness above `type(int256).max` before the cast in `OracleReceipt::price`. An integration that supports only a narrower market profile should also validate the Receipt's flavor and strike range before deploying the oracle.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. `OracleReceipt` now rejects moneyness values above the signed-integer range before casting them.

7.3.2 `OracleReceipt` retains an unused commented-out SCALE declaration

Description: `OracleReceipt` contains a commented-out SCALE constant declaration that is not part of the compiled pricing implementation. Retaining dead code beside active scale documentation can make it less clear which scaling factor the implementation actually uses.

Recommended Mitigation: Delete the commented-out declaration while retaining the explanatory comment about why the Morpho scale collapses to `1e18`.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. The unused commented SCALE declaration has been removed from `OracleReceipt`.

7.3.3 The public Black-Scholes helper does not document its parameter units

Description: `OracleReceipt::price(uint256,uint256,uint256,uint256,bool)` documents moneyness, but not the other parameters. `timeToExpiry` is expressed in seconds, while moneyness, `vol_`, and `rate_` are WAD-scaled. A direct caller can silently obtain the wrong result by assuming all numeric arguments use the same scale.

Recommended Mitigation: Add `NatSpec` for every parameter, explicitly stating that `timeToExpiry` is in seconds and that moneyness, `vol_`, and `rate_` are WAD-scaled.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. The Black-Scholes helper now documents the units and meaning of every parameter.

7.3.4 `OracleReceipt::price` can publish zero at its moneyness precision boundary

Description: `OracleReceipt::price` computes $m = \text{strike} * 1e18 / \text{spot}$ and then returns $(1e18 - \text{option-Price}) * 1e18$. At extreme moneyness, the WAD calculation can floor the Receipt value to zero. In particular, once `timeToExpiry == 0`, a call with $m == 0$ returns full intrinsic option value and the outer function publishes a zero Morpho price; before that boundary, `lnWad(0)` reverts instead.

Morpho treats a zero collateral price as zero borrowing capacity and permits collateral-denominated liquidation to compute zero repayment. However, outside the separately reported post-deadline valuation issue, this requires the modelled Receipt value to be below WAD resolution and is not credible for the intended WETH/sUSDe deployment profile. This is precision and fail-closed hardening, not a Low-severity loss scenario.

Recommended Mitigation: Handle the boundary explicitly: either calculate moneyness with enough precision to preserve a positive Receipt value or revert when the computation would publish zero. The post-deadline case should continue to be addressed by the separate lifecycle fix.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. OracleReceipt now reverts instead of publishing a zero collateral price.